

LeetCode 2454 - Next Greater Element IV

OFFICIAL DOCUMENTATION & ALGORITHM WALKTHROUGH

Problem Statement

Given an integer array `nums`, for every element `nums[i]`, find its second greater element.

A second greater element is defined as:

- There must exist a first greater element to the right of `nums[i]`.
- The second greater element is the first element after the first greater that is also greater than `nums[i]`.

If no such element exists, return -1.

Example

`nums = [2, 4, 0, 9, 6]`

Element	First Greater	Second Greater	Answer
2	4	9	9
4	9	6	6
0	9	6	6
9	—	—	-1
6	—	—	-1

Output

`[9,6,6,-1,-1]`

Observations

For a normal Next Greater Element, every element has only one stage.

Normal Next Greater Element Workflow



Waiting for Next Greater

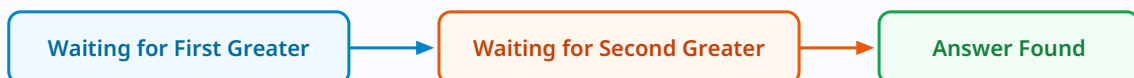


Answer Found

One monotonic stack is enough.

In this problem, every element has two stages.

LeetCode 2454 Two-Stage Workflow



Waiting for First Greater



Waiting for Second Greater



Answer Found

Since there are two waiting stages, one stack is not enough.

We need two stacks.

Core Intuition

Think of every element as a student.

Initially,

Student



Waiting for Interview Round 1

Once the first greater is found,

Student



Passed Round 1



Waiting for Interview Round 2

Once the second greater is found,

Student



Passed Round 2



Done

Student Interview Pipeline Analogy (Visual Mapping)

Initially:



Once the first greater is found:



Once the second greater is found:



Exactly the same idea is used in the algorithm.

Data Structures Used

- **Stack 1 (st1)**

Stores indices of elements

Waiting for FIRST Greater

- **Stack 2 (st2)**

Stores indices of elements

Already found FIRST Greater

Now waiting for SECOND Greater

- **Temporary Stack**

Used only while moving elements from

st1



st2

Its only purpose is to preserve the correct order inside st2 .

Why do we need the Temporary Stack?

Suppose

nums = [2, 4, 0, 9, 6]

Before processing 9,

```
st1
Top
0
4
```

Both 0 and 4 find their first greater (9).

If we directly push them into st2,

Pop 0

↓

st2

0

Pop 4

↓

st2

Top

4

0

Order becomes incorrect.

Instead,

move them into a temporary stack first.

st1

↓

temp

↓

st2

This reverses the order twice.

Final

st2

Top

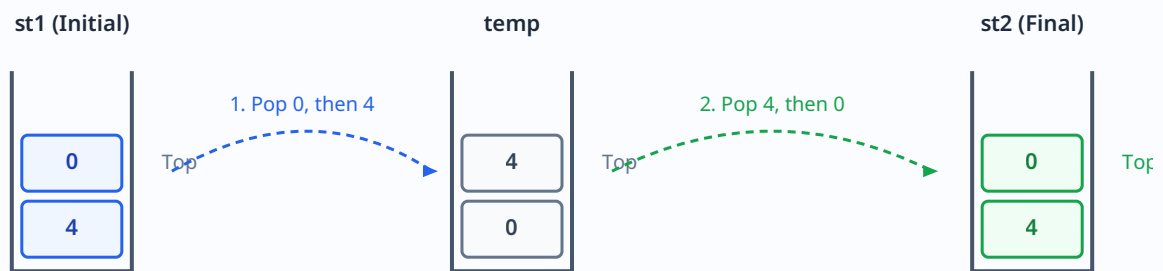
0

4

The processing order is preserved correctly.

Without this temporary stack, the algorithm may assign incorrect second greater elements.

Temporary Stack Order Double-Reversal Verification



Approach

Traverse the array from left to right.

For every element `nums[i]`:

Step 1

Resolve the second greater elements.

If the current element is greater than the elements waiting inside `st2`, then the current element becomes their second greater.

```
while (!st2.empty() && nums[i] > nums[st2.top()])
```

Store the answer and remove them.

Step 2

Find the first greater.

If the current element is greater than elements inside `st1`, then those elements have now found their first greater.

```
while (!st1.empty() && nums[i] > nums[st1.top()])
```

Remove them from st1.

Step 3

Move all popped elements

temp



st2

Now they are waiting for their second greater.

Step 4

Push the current index into `st1`.

The current element hasn't found its first greater yet.

Dry Run

Input

```
nums = [2, 4, 0, 9, 6]
```

Initially

```
st1 = []  
st2 = []  
  
ans = [-1, -1, -1, -1, -1]
```

Current = 2

Push into st1

```
st1  
2
```

Current = 4

4 > 2

Move 2

st1

4

st2

2

Current = 0

Push

st1

0

4

st2

2

Current = 9

Resolve st2

9 > 2

```
ans[0]=9
```

Remove 2.

Move from st1

Both

```
0  
4
```

found their first greater.

Move through temp.

Finally

```
st1  
9  
  
st2  
Top  
0  
4
```

Current = 6

Top of st2

0

$6 > 0$

ans[2]=6

Pop.

Next

4

$6 > 4$

ans[1]=6

Pop.

Push 6 into st1.

Final answer

[9,6,6,-1,-1]

Algorithm

```
Initialize answer with -1

Create Stack1
Create Stack2

for every element

    Resolve second greater
        while current > st2.top()

    Move first greater candidates
        while current > st1.top()

    Move temp → st2

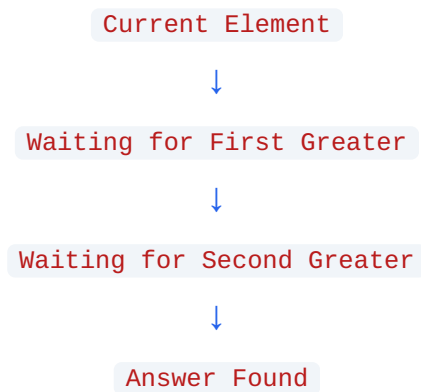
    Push current into st1

Return answer
```

Why does it work?

Every element moves through exactly three states.

Pipeline State Progression



The algorithm guarantees that every transition happens exactly once.
No element ever goes backwards.

Complexity

Time Complexity

$O(n)$

Reason:

Each index

- enters st1 once
- leaves st1 once
- enters st2 once
- leaves st2 once

Total operations are linear.

Space Complexity

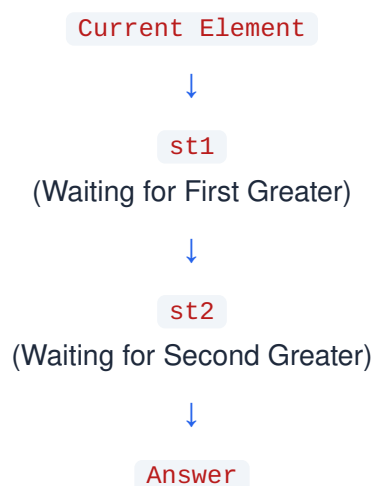
$O(n)$

Used by

- st1
- st2
- temp
- answer array

Key Intuition (Revision)

Remember only this pipeline:



And remember these 4 steps for every current element:

- Resolve everyone waiting for the second greater (st2).
- Move everyone who just found their first greater (st1 $\rightarrow$ temp).
- Move temp $\rightarrow$ st2 to preserve the correct order.
- Push the current index into st1.

If you remember this state transition, you can reconstruct the entire algorithm even if you forget the code.